Edtronaut Company Limited

SWE INTERN BACKEND ASSIGMENT

Live Code Execution System

Nguyen Minh Nhat

NhatDoo/MyStudyCase

March 2026

MỞ ĐẦU

Trong bối cảnh các nền tảng lập trình trực tuyến ngày càng phát triển, nhu cầu xây dựng một hệ thống có khả năng thực thi mã nguồn từ xa (Remote Code Execution) một cách an toàn và hiệu quả trở nên rất quan trọng. Tuy nhiên, việc cho phép người dùng chạy mã tùy ý trên server cũng đặt ra nhiều thách thức về bảo mật, hiệu năng và khả năng mở rộng. Báo cáo này trình bày quá trình thiết kế và triển khai hệ thống Code Execution Engine trong project MyStudyCase, tập trung vào kiến trúc bất đồng bộ sử dụng message queue, cơ chế cô lập bằng Docker, cùng các giải pháp đảm bảo độ tin cậy và khả năng mở rộng của hệ thống.

MỤC LỤC

1 Problem & Context 1
1.1 Problem 1
1.2 Context 1
2 Solution Review 2
2.1 Thành phần chính: 2
2.2 Luồng hoạt động: 3
3 Architecture diagram 3
3.1 Các thành phần chính: 3
3.2 Luồng xử lý 4
4 System Flow (Async Execution) 4
4.1 Luồng xử lý chính 5
4.2 Start machinne 5
5 API Design 6
5.1 Core API 6
5.2 Design Decisions 6
5.3 Basic Constraints 6
5 Database design 7
6.1 Core Table 7
6.2 Relationships 7
6.3 Schema Overeview 7
6 Execution lifecycle 7
6.1 Flow Review 7
6.2 Key Design 8
7 Queue + worker 8
8 Safety & security 9
9 Scalability 10
10 Trade-offs 10
11 Future improvements 11

1. Problem & Context

1.1. Problem

- Các nền tảng mô phỏng công việc (Job Simulation) hiện nay hướng tới việc tái hiện môi trường làm việc thực tế.

Tuy nhiên, đa số hệ thống chỉ hỗ trợ:

• Nhập văn bản (text)

• Upload file

Điều này gây ra các hạn chế:

• Không hỗ trợ thực thi code trực tiếp

• Trải nghiệm kém cho các vị trí kỹ thuật

• Không thể kiểm tra tính đúng sai của code theo thời gian thực

• Thiếu tính tương tác trong quá trình học

Về mặt hệ thống:

• Thực thi đồng bộ (synchronous) dễ gây block server

• Khó xử lý nhiều người dùng cùng lúc

• Không có cơ chế cách ly code người dùng

• Dễ bị lạm dụng tài nguyên (infinite loop, quá tải CPU/RAM)

1.2. Context

- Để giải quyết các vấn đề trên, cần xây dựng một hệ thống backend hỗ trợ thực thi code trực tiếp trong nền tảng Job Simulation.

Hệ thống cần cho phép người dùng:

- Viết và cập nhật code theo thời gian thực
- Thực thi code một cách bất đồng bộ
- Nhận kết quả (stdout, stderr, trạng thái)
- Hỗ trợ nhiều người dùng đồng thời

Đồng thời đảm bảo:

- Hiệu năng cao (không block request)
- Khả năng mở rộng (scalability)
- An toàn (giới hạn tài nguyên, chống lạm dụng)
- Độ tin cậy (theo dõi vòng đời execution)

2. Solution Review

MyStudyCase là một hệ thống demo giúp:

- Thực thi test song song (parallel testing)
- Tích hợp database với Prisma ORM
- Chuẩn hóa môi trường bằng Docker
- Áp dụng best practices trong Node.js

2.1. Thành phần chính:

• Test Engine (Jest + Parallel Execution)

→ Chạy nhiều test cùng lúc

• Database Layer (Prisma ORM)

→ Query type-safe, migration dễ dàng

• Docker Environment

→ Môi trường đồng nhất, dễ setup

• TypeScript

→ Đảm bảo type safety & maintainability

2.2. Luồng hoạt động:

1. Developer viết test

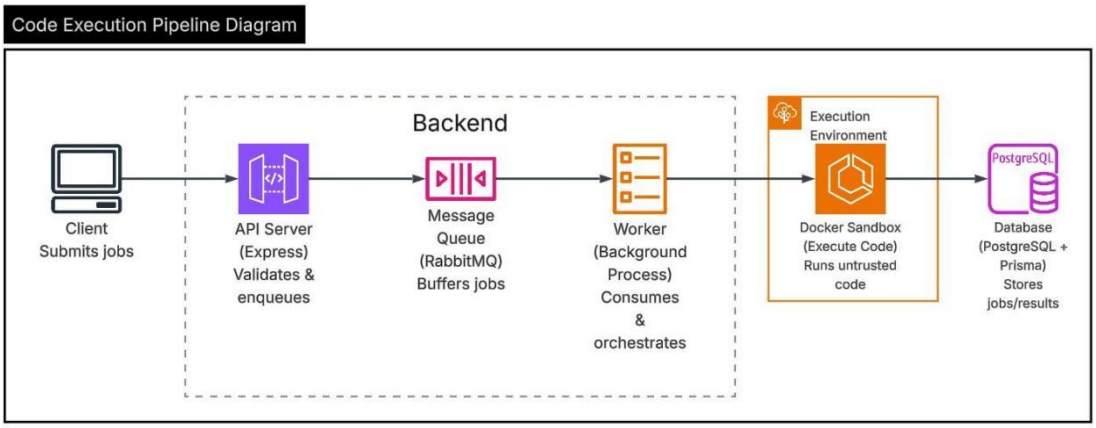
2. Jest chia test thành nhiều worker

3. Các worker chạy song song

4. Kết nối database qua Prisma

5. Trả về kết quả + metrics

3. Architecture diagram



3.1. Các thành phần chính:

• API Server

- Nhận request & validate

- Không xử lý nặng (non-blocking)

• RabbitMQ (Queue)

- Lưu job execution

- Đảm bảo reliability (ACK)

• Worker

- Xử lý job nền
- Thực thi code

• Docker Sandbox

- Isolated environment
- Giới hạn CPU / RAM / Network

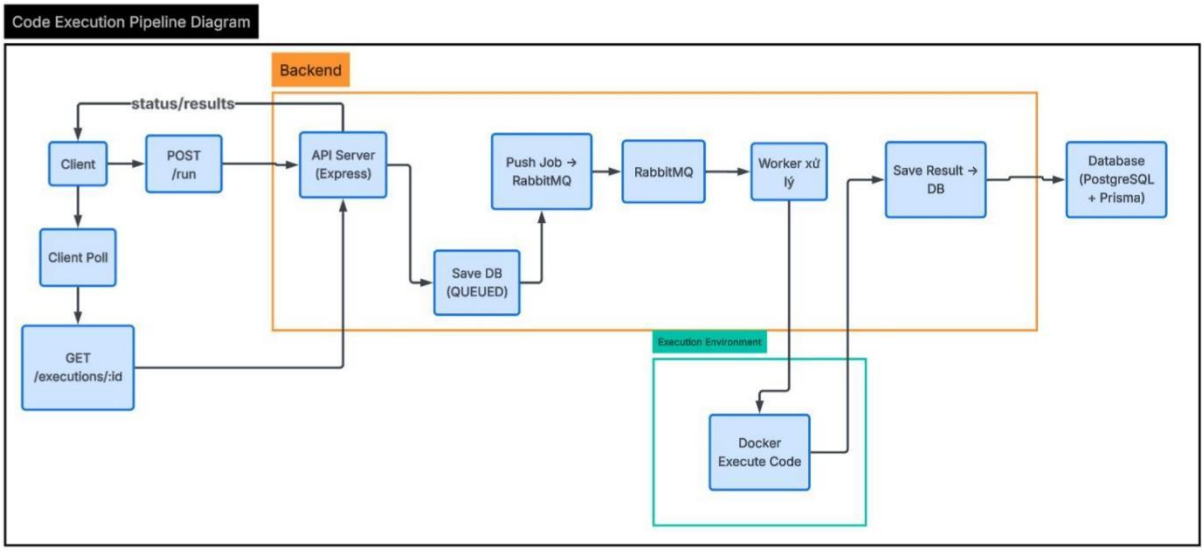
• Database (Prisma)

- Lưu session, execution, log

3.2. Luồng xử lý

1. Client gửi request run code
2. API lưu DB + đẩy job vào queue
3. Worker lấy job từ queue
4. Chạy code trong Docker sandbox
5. Lưu kết quả vào database
6. Client lấy kết quả (polling)

4. System Flow (Async Execution)



4.1. Luồng xử lý chính

1. User gửi request chạy code

2. API validate + lưu DB (status = QUEUED)

3. Đẩy job vào RabbitMQ

4. Worker lấy job xử lý

5. Chạy code trong Docker sandbox

6. Lưu kết quả (stdout, stderr, time)

7. Client polling để lấy kết quả

4.2. Start machinne

QUEUED → WAITING → RUNNING → COMPLETED / FAILED / TIMEOUT

- QUEUED: vừa nhận job
- WAITING: chuẩn bị môi trường
- RUNNING: đang execute
- COMPLETED: thành công

- FAILED: lỗi runtime
- TIMEOUT: quá thời gian

5 API Design

5.1. Core API

POST /code-sessions → Tạo session

PATCH /code-sessions/:id → Auto-save code

POST /code-sessions/:id/run → Execute code (async)

GET /executions/:id → Lấy kết quả

Async Execution Flow

Client → POST /run → 202 Accepted

→ nhận execution_id

→ polling GET /executions/:id

5.2.Design Decisions

• RESTful API (resource-based)

• Async processing (RabbitMQ)

• Non-blocking (tránh timeout)

• Polling-based result retrieval

5.3. Basic Constraints

• Rate limit: 10 req/min (run endpoint)

• Input validation

• HTTP status chuẩn (201, 202, 200, 404 , 500)

6. Database design

6.1. Core Table

• CodeSession , Execution , ExecutionLog

6.2. Relationships

CodeSession (1) → (N) Execution

Execution (1) → (N) ExecutionLog

6.3. Schema Overeview

CodeSession

id (PK) , language ,sourceCode , status (ACTIVE | CLOSED) ,createdAt, updatedAt

Execution

id (PK) , sessionId (FK) ,status ,stdout, stderr ,executionTimeMs ,startedAt, completedAt

ExecutionLog

id (PK) , executionId (FK) ,status ,message ,createdAt

7. Execution lifecycle

7.1 Flow Review

1. Client gọi API (POST /run)

2. Tạo execution (QUEUED)

3. Push job vào RabbitMQ

4. Worker xử lý job

5. Execute code trong Docker

6. Lưu kết quả & trả về client

7.2. Key Design

• Non-blocking API

• Queue-based processing

• Retry + Dead Letter Queue

• Execution isolation (Docker)

8. Queue + worker

- Hệ thống sử dụng mô hình asynchronous processing nhằm tách biệt giữa API và quá trình thực thi code. Thay vì xử lý trực tiếp, API chỉ đóng vai trò tiếp nhận request, lưu dữ liệu và gửi job vào message queue. Việc xử lý sẽ được thực hiện bởi các worker độc lập.

- Cụ thể, khi client gửi yêu cầu thực thi code, API sẽ tạo một bản ghi `Execution` với trạng thái ban đầu là `QUEUED`, sau đó gửi `executionId` vào hàng đợi RabbitMQ. Worker sẽ lấy job từ queue (với cơ chế prefetch để xử lý song song), cập nhật trạng thái sang `WAITING` và `RUNNING`, sau đó thực thi code trong môi trường Docker sandbox. Khi hoàn tất, kết quả (stdout, stderr, thời gian thực thi) sẽ được lưu lại và trạng thái được cập nhật thành `COMPLETED`, `FAILED` hoặc `TIMEOUT`.

- Hệ thống queue được cấu hình theo hướng đảm bảo độ tin cậy cao . Các job được lưu trong durable queue để tránh mất dữ liệu khi hệ thống restart. Ngoài ra, cơ chế retry (tối đa 3 lần) kết hợp với Dead Letter Queue (DLQ) giúp xử lý các job lỗi mà không làm ảnh hưởng đến luồng chính. Trong trường hợp worker bị crash, job chưa được acknowledge sẽ tự động được đưa lại vào queue để worker khác xử lý.

- Worker được thiết kế với khả năng tự phục hồi và vận hành ổn định , bao gồm auto reconnect khi mất kết nối, graceful shutdown khi hệ thống dừng,

và xử lý idempotent để tránh thực thi trùng lặp. Điều này giúp hệ thống hoạt động bền vững trong môi trường production.

- Để đảm bảo an toàn khi thực thi code từ người dùng, hệ thống sử dụng Docker sandbox với các giới hạn nghiêm ngặt như chặn network (`--network=none`), giới hạn CPU và bộ nhớ. Nhờ đó, hệ thống có thể ngăn chặn các hành vi tấn công hoặc sử dụng tài nguyên vượt mức.

- Kiến trúc này mang lại nhiều lợi ích quan trọng: API không bị block khi xử lý các tác vụ nặng, hệ thống có khả năng mở rộng bằng cách tăng số lượng worker, đồng thời đảm bảo khả năng chịu lỗi thông qua cơ chế retry và queue trung gian.

9. Safety & security

- Hệ thống Code Execution Engine tiềm ẩn rủi ro cao do cho phép thực thi mã tùy ý, tuy nhiên đã áp dụng nhiều cơ chế bảo vệ quan trọng.

- Trước hết, việc sử dụng Docker sandbox giúp cô lập môi trường thực thi. Các container được cấu hình với --network=none để chặn truy cập mạng, cùng với giới hạn tài nguyên (--memory=128m, --cpus=0.5) nhằm ngăn chặn các tấn công tiêu tốn tài nguyên. Ngoài ra, cơ chế timeout 10 giây và tự động xóa container sau khi chạy giúp hạn chế vòng lặp vô hạn và tránh để lại dữ liệu dư thừa.

- Hệ thống cũng áp dụng rate limiting (10 request/phút/IP) để giảm nguy cơ abuse. File thực thi được tạo với tên ngẫu nhiên (UUID) và xóa ngay sau khi hoàn tất, đảm bảo tính cô lập và tránh rò rỉ dữ liệu.

- Về bảo mật thông tin, các biến môi trường được loại trừ khỏi Git và Docker image, hạn chế rò rỉ secrets. Đồng thời, hệ thống sử dụng logging có cấu trúc giúp dễ dàng theo dõi và xử lý sự cố.

- Nhìn chung, hệ thống đã đảm bảo các nguyên tắc cơ bản về isolation, resource control và fault protection, phù hợp cho môi trường thực thi code an toàn ở mức cơ bản.

10. Scalability

- Hệ thống được thiết kế theo mô hình Producer–Consumer với message queue, giúp tách biệt giữa API và quá trình xử lý, từ đó hỗ trợ mở rộng hiệu quả.

- Thành phần quan trọng nhất là RabbitMQ, đóng vai trò trung gian giúp hệ thống dễ dàng scale theo chiều ngang. API server chỉ enqueue job mà không cần chờ xử lý, trong khi các worker có thể được nhân bản để tăng throughput. Khi tải tăng cao, queue sẽ đóng vai trò buffer (backpressure), giúp hệ thống không bị quá tải đột ngột.

- Worker hỗ trợ xử lý song song thông qua cơ chế prefetch (5 jobs), cho phép kiểm soát concurrency tùy theo tài nguyên máy. Ngoài ra, cơ chế retry kết hợp với Dead Letter Queue (DLQ) đảm bảo các job lỗi không làm gián đoạn toàn bộ hệ thống, tăng tính ổn định khi scale.

- Hệ thống cũng có khả năng phục hồi tốt nhờ cơ chế auto reconnect với exponential backoff, giúp worker tự động kết nối lại khi RabbitMQ gặp sự cố mà không gây quá tải.

- Về tầng dữ liệu, việc sử dụng Supabase (managed PostgreSQL) giúp giảm đáng kể độ phức tạp khi mở rộng database, nhờ hỗ trợ sẵn replication, backup và connection pooling.

- Tổng thể, kiến trúc hiện tại cho phép scale theo chiều ngang ở tầng worker, đồng thời đảm bảo hệ thống vẫn ổn định dưới tải cao. Tuy nhiên, các điểm như API stateless scaling, queue partitioning hoặc monitoring nâng cao có thể được cải thiện thêm trong tương lai.

11. Trade-offs

- Trong quá trình thiết kế, hệ thống đã đưa ra một số lựa chọn đánh đổi giữa hiệu năng, bảo mật và độ phức tạp.

- Trước hết, việc chọn asynchronous execution (queue-based) thay vì synchronous là quyết định phù hợp, giúp hệ thống không bị block và dễ dàng mở rộng. Tuy nhiên, điều này làm tăng độ phức tạp trong việc theo dõi trạng thái và xử lý lỗi.

- Về môi trường thực thi, việc sử dụng native execution thay vì Docker giúp đơn giản hóa triển khai và giảm overhead, nhưng đánh đổi lớn về mặt bảo mật do thiếu cơ chế isolation mạnh. Đây là điểm cần cải thiện nếu đưa vào production.

- Hệ thống sử dụng managed database (Supabase) thay vì self-hosted, giúp giảm chi phí vận hành và tăng độ ổn định, phù hợp với giai đoạn prototype. Tuy nhiên, sẽ bị hạn chế quyền kiểm soát sâu khi cần tối ưu hiệu năng.

- Ở tầng giao tiếp, việc sử dụng polling thay vì WebSocket đơn giản hơn để triển khai nhưng mang lại trải nghiệm kém mượt và tốn tài nguyên hơn khi số lượng client tăng.

- Ngoài ra, kiến trúc monorepo , manual dependency injection và cách tiếp cận hybrid ORM (Prisma + raw SQL) đều là các lựa chọn hợp lý ở quy mô hiện tại, cân bằng giữa tốc độ phát triển và khả năng kiểm soát hệ thống.

- Cuối cùng, việc cấu hình CORS mở hoàn toàn giúp thuận tiện trong phát triển nhưng tiềm ẩn rủi ro bảo mật và cần được giới hạn lại trong môi trường production.

12. Future improvements

- Hệ thống hiện tại đã có nền tảng tốt, tuy nhiên vẫn còn nhiều hướng cải thiện để tiến tới production-ready.

- Trong ngắn hạn, cần bổ sung Authentication & Authorization thông qua API Key để kiểm soát truy cập và hỗ trợ các tính năng như rate limit theo user,

quota và session ownership. Đồng thời, việc thêm User model vào database là cần thiết để quản lý người dùng và mở rộng hệ thống sau này.

- Về trải nghiệm, có thể thay polling bằng Server-Sent Events (SSE) để cung cấp cập nhật trạng thái execution theo thời gian thực, giúp giảm tải cho server và cải thiện UX. Ngoài ra, việc sử dụng Redis cho rate limiting sẽ đảm bảo hoạt động chính xác khi scale nhiều instance API.

- Hệ thống cũng có thể mở rộng dễ dàng bằng cách hỗ trợ thêm nhiều ngôn ngữ lập trình thông qua Docker executor, tận dụng kiến trúc hiện tại mà không cần thay đổi lớn.

- Trong dài hạn, cần xây dựng observability stack (logging, metrics, tracing) để giám sát hệ thống, cùng với cơ chế auto-scaling worker dựa trên queue depth nhằm tối ưu tài nguyên. Bên cạnh đó, việc bổ sung execution analytics và hệ thống quota theo plan (FREE/PRO) sẽ giúp hệ thống tiến gần hơn tới một sản phẩm hoàn chỉnh.